MODULE 5: BASIC PROCESSING UNIT

SOME FUNDAMENTAL CONCEPTS

• To execute an instruction, processor has to perform following 3 steps:

1) Fetch contents of memory-location pointed to by PC. Content of this location is an instruction to be executed. The instructions are loaded into IR, Symbolically, this operation is written as:
   IR← [[PC]]
2) Increment PC by 4.
   PC← [PC] +4
3) Carry out the actions specified by instruction (in the IR).

• The first 2 steps are referred to as **Fetch Phase**.
   Step 3 is referred to as **Execution Phase**.

• The operation specified by an instruction can be carried out by performing one or more of the following actions:

1) Read the contents of a given memory-location and load them into a register.
2) Read data from one or more registers.
3) Perform an arithmetic or logic operation and place the result into a register.
4) Store data from a register into a given memory-location.

• The hardware-components needed to perform these actions are shown in Figure 5.1.

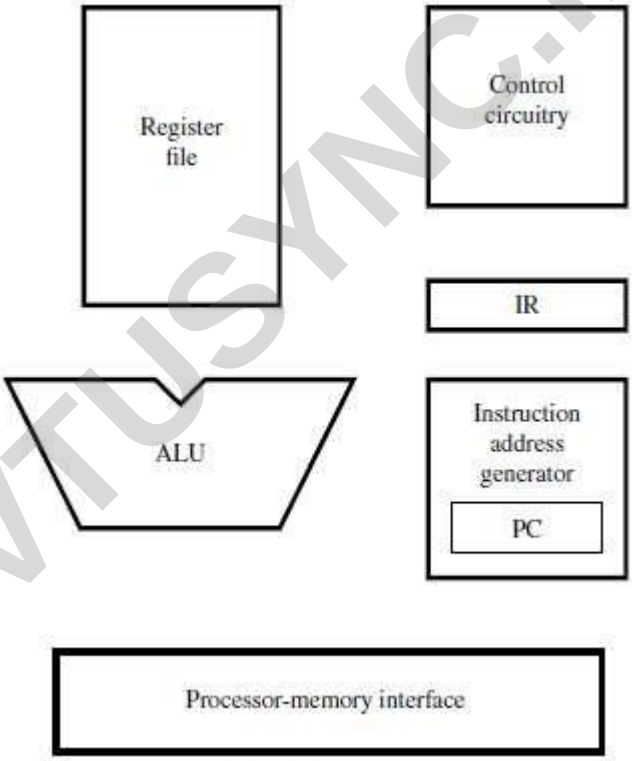


Figure 5.1 Main hardware components of a processor.

SINGLE BUS ORGANIZATION

- ALU and all the registers are interconnected via a **Single Common Bus** (Figure 7.1).
- Data & address lines of the external memory-bus is connected to the internal processor-bus via MDR & MAR respectively. (MDR→ Memory Data Register, MAR → Memory Address Register).
- **MDR** has 2 inputs and 2 outputs. Data may be loaded
  - → into MDR either from memory-bus (external) or
  - → from processor-bus (internal).
- **MAR**'s input is connected to internal-bus; MAR's output is connected to external-bus.
- **Instruction Decoder & Control Unit** is responsible for
  - → issuing the control-signals to all the units inside the processor.
  - → implementing the actions specified by the instruction (loaded in the IR).
- Register R0 through R(n-1) are the **Processor Registers**. The programmer can access these registers for general-purpose use.
- Only processor can access 3 registers **Y**, **Z** & **Temp** for temporary storage during program-execution. The programmer cannot access these 3 registers.
- In **ALU**, 1) „A" input gets the operand from the output of the multiplexer (MUX).
  2) „B" input gets the operand directly from the processor-bus.
- There are 2 options provided for „A" input of the ALU.
- MUX is used to select one of the 2 inputs.
- **MUX** selects either
  - → output of Y or
  - → constant-value 4(which is used to increment PC content).

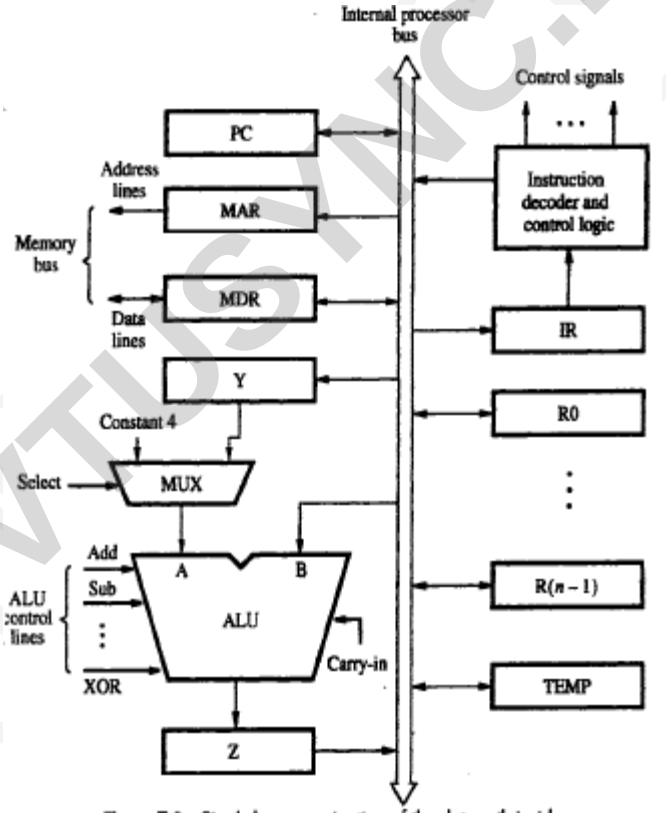


Figure 7.1 Single-bus organization of the datapath inside a processor.

- An instruction is executed by performing one or more of the following operations:
  1) Transfer a word of data from one register to another or to the ALU.
  2) Perform arithmetic or a logic operation and store the result in a register.
  3) Fetch the contents of a given memory-location and load them into a register.
  4) Store a word of data from a register into a given memory-location.
- **Disadvantage:** Only one data-word can be transferred over the bus in a clock cycle.

  Solution: Provide multiple internal-paths. Multiple paths allow several data-transfers to take place in parallel.

REGISTER TRANSFERS

• Instruction execution involves a sequence of steps in which data are transferred from one register to another.

• For each register, two control-signals are used: Ri_{in} & Ri_{out}. These are called **Gating Signals.**

• Ri_{in}=1 → data on bus is loaded into Ri.

Ri_{out}=1 → content of Ri is placed on bus.

Ri_{out}=0, → bus can be used for transferring data from other registers.

• For example, *Move R1, R2;* This transfers the contents of register R1 to register R2. This can be accomplished as follows:

1) Enable the output of registers R1 by setting $R1_{out}$ to 1 (Figure 7.2).
   This places the contents of R1 on processor-bus.
2) Enable the input of register R2 by setting $R2_{out}$ to 1.
   This loads data from processor-bus into register R4.

• All operations and data transfers within the processor take place within time-periods defined by the **processor-clock**.

• The control-signals that govern a particular transfer are asserted at the start of the clock cycle.

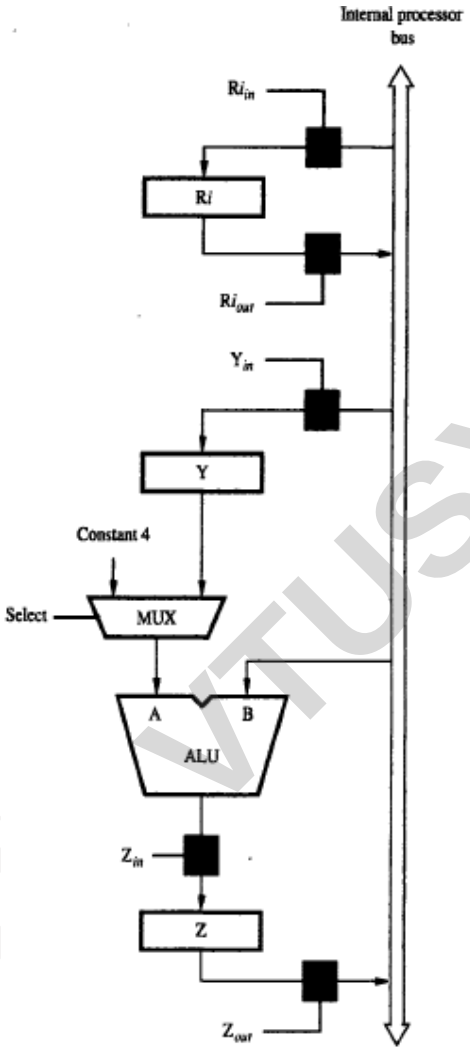


Figure 7.2 Input and output gating for the registers in Figure 7.1.

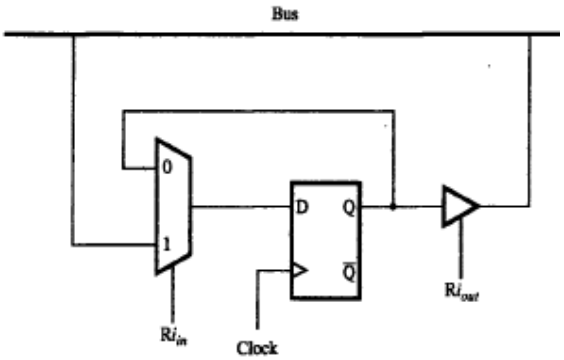


Figure 7.3 Input and output gating for one register bit.

Input & Output Gating for one Register Bit

• A 2-input multiplexer is used to select the data applied to the input of an edge-triggered D flip-flop.

• Ri_{in}=1 → mux selects data on bus. This data will be loaded into flip-flop at rising-edge of clock.

Ri_{in}=0 → mux feeds back the value currently stored in flip-flop (Figure 7.3).

• Q output of flip-flop is connected to bus via a tri-state gate.

Ri_{out}=0 → gate's output is in the high-impedance state.

Ri_{out}=1 → the gate drives the bus to 0 or 1, depending on the value of Q.

PERFORMING AN ARITHMETIC OR LOGIC OPERATION

- The ALU performs arithmetic operations on the 2 operands applied to its A and B inputs.
- One of the operands is output of MUX;
  And, the other operand is obtained directly from processor-bus.
- The result (produced by the ALU) is stored temporarily in register Z.
- The sequence of operations for [R3]←[R1]+[R2] is as follows:
  1) $R1_{out}$, Y_{in}
  2) $R2_{out}$, SelectY, Add, Z_{in}
  3) Z_{out}, $R3_{in}$
- Instruction execution proceeds as follows:
  Step 1 --> Contents from register R1 are loaded into register Y.
  Step2 --> Contents from Y and from register R2 are applied to the A and B inputs of ALU;
  Addition is performed &
  Result is stored in the Z register.
  Step 3 --> The contents of Z register is stored in the R3 register.
- The signals are activated for the duration of the clock cycle corresponding to that step. All other signals are inactive.

CONTROL-SIGNALS OF MDR

- The MDR register has 4 control-signals (Figure 7.4):
  1) MDR_{in} & MDR_{out} control the connection to the internal processor data bus &
  2) MDR_{inE} & MDR_{outE} control the connection to the memory Data bus.
- MAR register has 2 control-signals.
  1) MAR_{in} controls the connection to the internal processor address bus &
  2) MAR_{out} controls the connection to the memory address bus.

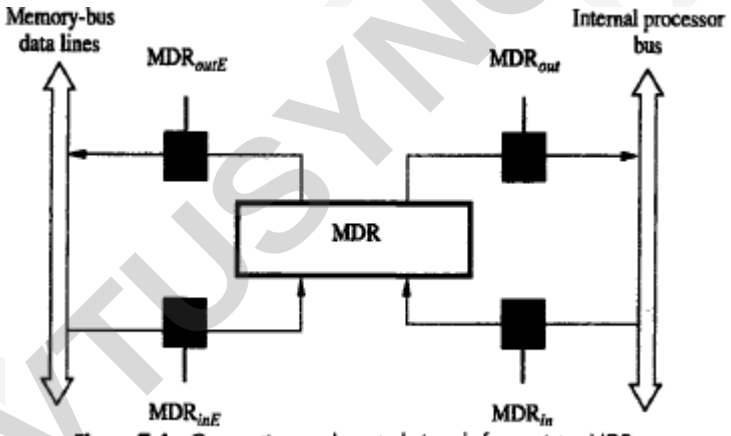


Figure 7.4 Connection and control signals for register MDR.

FETCHING A WORD FROM MEMORY

• To fetch instruction/data from memory, processor transfers required address to MAR.
At the same time, processor issues Read signal on control-lines of memory-bus.

• When requested-data are received from memory, they are stored in MDR. From MDR, they are transferred to other registers.

• The response time of each memory access varies (based on cache miss, memory-mapped I/O). To accommodate this, MFC is used. (MFC → Memory Function Completed).

• MFC is a signal sent from addressed-device to the processor. MFC informs the processor that the requested operation has been completed by addressed-device.

• Consider the instruction Move (R1),R2. The sequence of steps is (Figure 7.5):

1) $R1_{out}$, MAR_{in}, Read ;desired address is loaded into MAR & Read command is issued.
2) MDR_{inE}, WMFC ;load MDR from memory-bus & Wait for MFC response from memory.
3) MDR_{out}, $R2_{in}$;load R2 from MDR.

where WMFC=control-signal that causes processor's control. circuitry to wait for arrival of MFC signal.

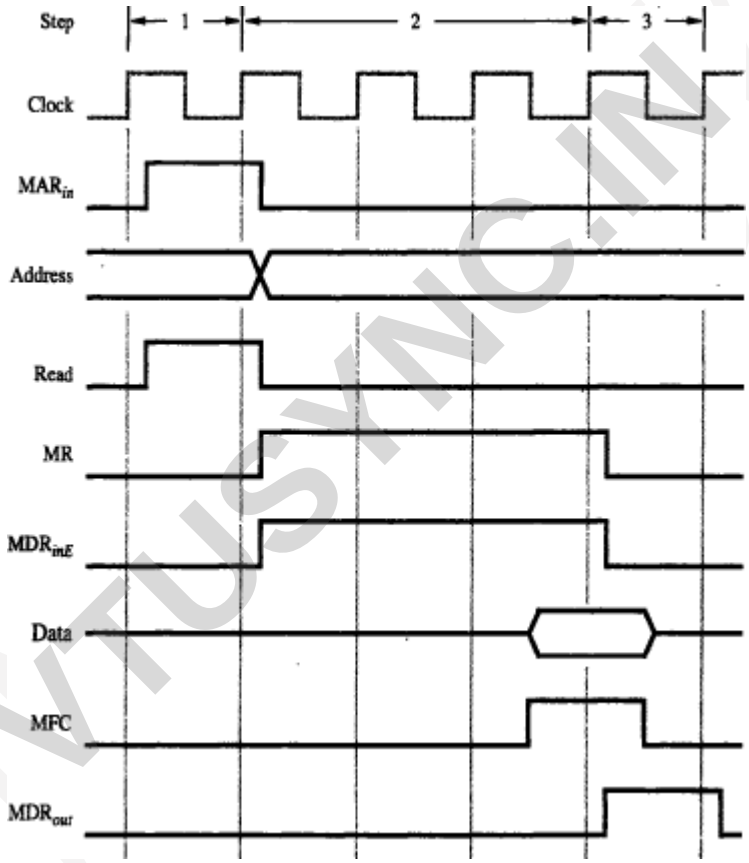


Figure 7.5 Timing of a memory Read operation.

Storing a Word in Memory

• Consider the instruction *Move R2,(R1).* This requires the following sequence:

1) $R1_{out}$, MAR_{in} ;desired address is loaded into MAR.
2) $R2_{out}$, MDR_{in}, Write ;data to be written are loaded into MDR & Write command is issued.
3) MDR_{outE}, WMFC ;load data into memory-location pointed by R1 from MDR.

EXECUTION OF A COMPLETE INSTRUCTION

• Consider the instruction *Add (R3),R1* which adds the contents of a memory-location pointed by R3 to register R1. Executing this instruction requires the following actions:

1) Fetch the instruction.
2) Fetch the first operand.
3) Perform the addition &
4) Load the result into R1.

Step	Action
1	PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2	Z_{out}, PC_{in}, Y_{in}, WMFC
3	MDR_{out}, IR_{in}
4	$R3_{out}$, MAR_{in}, Read
5	$R1_{out}$, Y_{in}, WMFC
6	MDR_{out}, SelectY, Add, Z_{in}
7	Z_{out}, $R1_{in}$, End

Figure 7.6 Control sequence for execution of the instruction Add (R3),R1

• Instruction execution proceeds as follows:

Step1--> The instruction-fetch operation is initiated by
→ loading contents of PC into MAR &
→ sending a Read request to memory.
The Select signal is set to Select4, which causes the Mux to select constant 4. This value is added to operand at input B (PC"s content), and the result is stored in Z.

Step2--> Updated value in Z is moved to PC. This completes the PC increment operation and PC will now point to next instruction.

Step3--> Fetched instruction is moved into MDR and then to IR.
The step 1 through 3 constitutes the **Fetch Phase**.
At the beginning of step 4, the instruction decoder interprets the contents of the IR. This enables the control circuitry to activate the control-signals for steps 4 through 7.
The step 4 through 7 constitutes the **Execution Phase**.

Step4--> Contents of R3 are loaded into MAR & a memory read signal is issued.

Step5--> Contents of R1 are transferred to Y to prepare for addition.

Step6--> When Read operation is completed, memory-operand is available in MDR, and the addition is performed.

Step7--> Sum is stored in Z, then transferred to R1.The End signal causes a new instruction fetch cycle to begin by returning to step1.

BRANCHING INSTRUCTIONS

• Control sequence for an **unconditional branch instruction** is as follows:

Step	Action
1	PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2	Z_{out}, PC_{in}, Y_{in}, WMFC
3	MDR_{out}, IR_{in}
4	Offset-field-of-IR_{out}, Add, Z_{in}
5	Z_{out}, PC_{in}, End

Figure 7.7 Control sequence for an unconditional Branch instruction.

• Instruction execution proceeds as follows:

Step 1-3--> The processing starts & the fetch phase ends in step3.

Step 4--> The offset-value is extracted from IR by instruction-decoding circuit.

Since the updated value of PC is already available in register Y, the offset X is gated onto the bus, and an addition operation is performed.

Step 5--> the result, which is the branch-address, is loaded into the PC.

• The branch instruction loads the branch target address in PC so that PC will fetch the next instruction from the branch target address.

• The branch target address is usually obtained by adding the offset in the contents of PC.

• The offset X is usually the difference between the branch target-address and the address immediately following the branch instruction.

• In case of **conditional branch**,

we have to check the status of the condition-codes before loading a new value into the PC.

e.g.: Offset-field-of-IR_{out}, Add, Z_{in}, If N=0 then End

If N=0, processor returns to step 1 immediately after step 4.

If N=1, step 5 is performed to load a new value into PC.

MULTIPLE BUS ORGANIZATION

• **Disadvantage of Single-bus organization:** Only one data-word can be transferred over the bus in a clock cycle. This increases the steps required to complete the execution of the instruction

Solution: To reduce the number of steps, most processors provide multiple internal-paths. Multiple paths enable several transfers to take place in parallel.

• As shown in fig 7.8, three buses can be used to connect registers and the ALU of the processor.

• All general-purpose registers are grouped into a single block called the **Register File**.

• Register-file has 3 ports:

1) Two output-ports allow the contents of 2 different registers to be simultaneously placed on buses A & B.

2) Third input-port allows data on bus C to be loaded into a third register during the same clock-cycle.

• Buses A and B are used to transfer source-operands to A & B inputs of ALU.

• The result is transferred to destination over bus C.

• **Incrementer Unit** is used to increment PC by 4.

Step	Action
1	PC_{out}, R=B, MAR_{in}, Read, IncPC
2	WMFC
3	MDR_{outB}, R=B, IR_{in}
4	$R4_{outA}$, $R5_{outB}$, SelectA, Add, $R6_{in}$, End

Figure 7.9 Control sequence for the instruction Add R4,R5,R6

• Instruction execution proceeds as follows:

Step 1--> Contents of PC are

→ passed through ALU using R=B control-signal &

→ loaded into MAR to start memory Read operation. At the same time, PC is incremented by 4.

Step2--> Processor waits for MFC signal from memory.

Step3--> Processor loads requested-data into MDR, and then transfers them to IR.

Step4--> The instruction is decoded and add operation takes place in a single step.

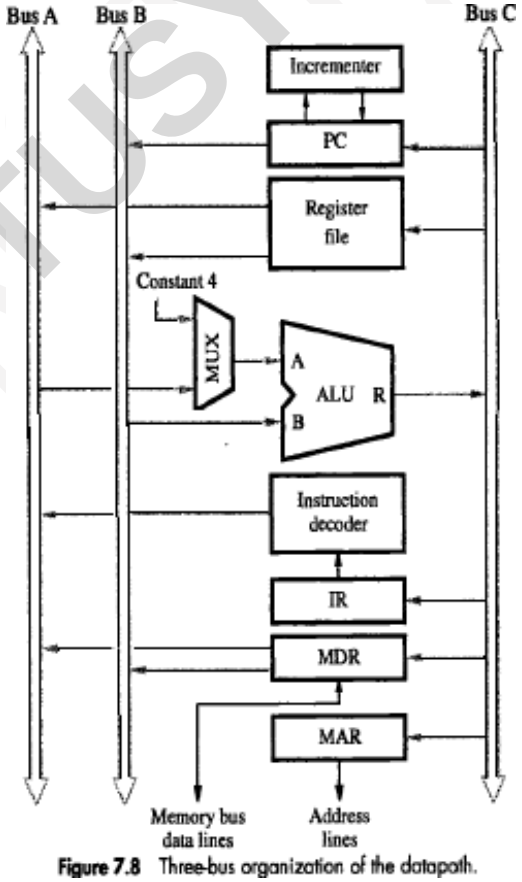


Figure 7.8 Three-bus organization of the datapath.

COMPLETE PROCESSOR

- This has separate processing-units to deal with integer data and floating-point data.
  Integer Unit → To process integer data. (Figure 7.14).
  Floating Unit → To process floating –point data.
- **Data-Cache** is inserted between these processing-units & main-memory.
  The integer and floating unit gets data from data cache.
- **Instruction-Unit** fetches instructions
  → from an instruction-cache or
  → from main-memory when desired instructions are not already in cache.
- Processor is connected to system-bus &
  hence to the rest of the computer by means of a **Bus Interface.**
- Using separate caches for instructions & data is common practice in many processors today.
- A processor may include several units of each type to increase the potential for concurrent operations.
- The 80486 processor has 8-kbytes single cache for both instruction and data.
  Whereas the Pentium processor has two separate 8 kbytes caches for instruction and data.

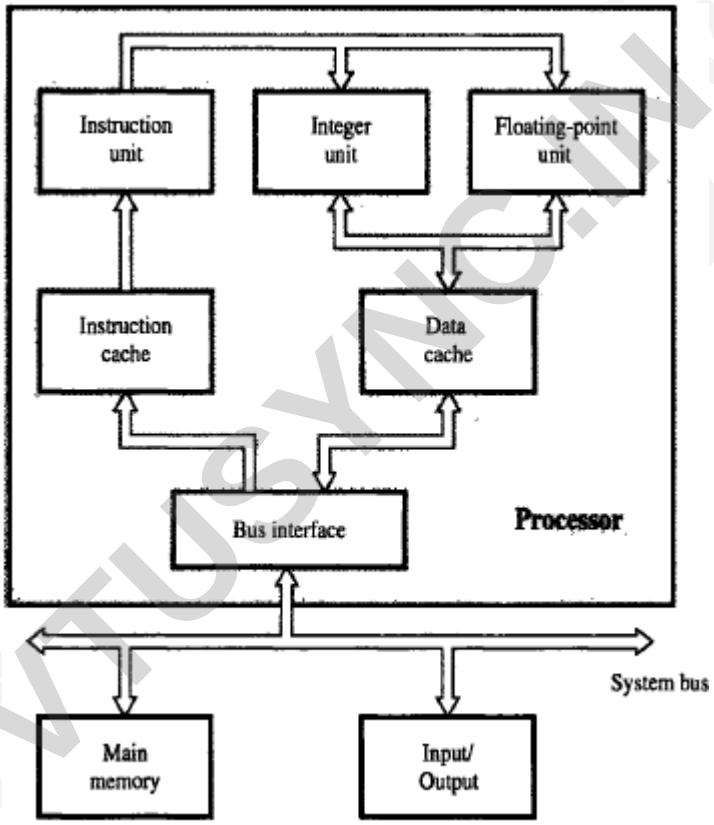


Figure 7.14 Block diagram of a complete processor.

Note:
To execute instructions, the processor must have some means of generating the control-signals. There are two approaches for this purpose:
1) Hardwired control and 2) Microprogrammed control.

HARDWIRED CONTROL

- Hardwired control is a method of control unit design (Figure 7.11).
- The control-signals are generated by using logic circuits such as gates, flip-flops, decoders etc.
- **Decoder/Encoder Block** is a combinational-circuit that generates required control-outputs depending on state of all its inputs.
- **Instruction Decoder**
  - It decodes the instruction loaded in the IR.
  - If IR is an 8 bit register, then instruction decoder generates 2^8(256 lines); one for each instruction.
  - It consists of a separate output-lines INS_1 through INS_m for each machine instruction.
  - According to code in the IR, one of the output-lines INS_1 through INS_m is set to 1, and all other lines are set to 0.
- **Step-Decoder** provides a separate signal line for each step in the control sequence.
- **Encoder**
  - It gets the input from instruction decoder, step decoder, external inputs and condition codes.
  - It uses all these inputs to generate individual control-signals: Y_{in}, PC_{out}, Add, End and so on.
  - For example (Figure 7.12), $Z_{in}=T_1+T_6.ADD+T_4.BR$
    ;This signal is asserted during time-slot T_1 for all instructions.
    during T_6 for an Add instruction.
    during T_4 for unconditional branch instruction
- When **RUN**=1, counter is incremented by 1 at the end of every clock cycle.
  When RUN=0, counter stops counting.
- After execution of each instruction, **end** signal is generated. End signal resets step counter.
- Sequence of operations carried out by this machine is determined by wiring of logic circuits, hence the name "**hardwired**".
- **Advantage**: Can operate at high speed.
- **Disadvantages:**
  1) Since no. of instructions/control-lines is often in hundreds, the complexity of control unit is very high.
  2) It is costly and difficult to design.
  3) The control unit is inflexible because it is difficult to change the design.

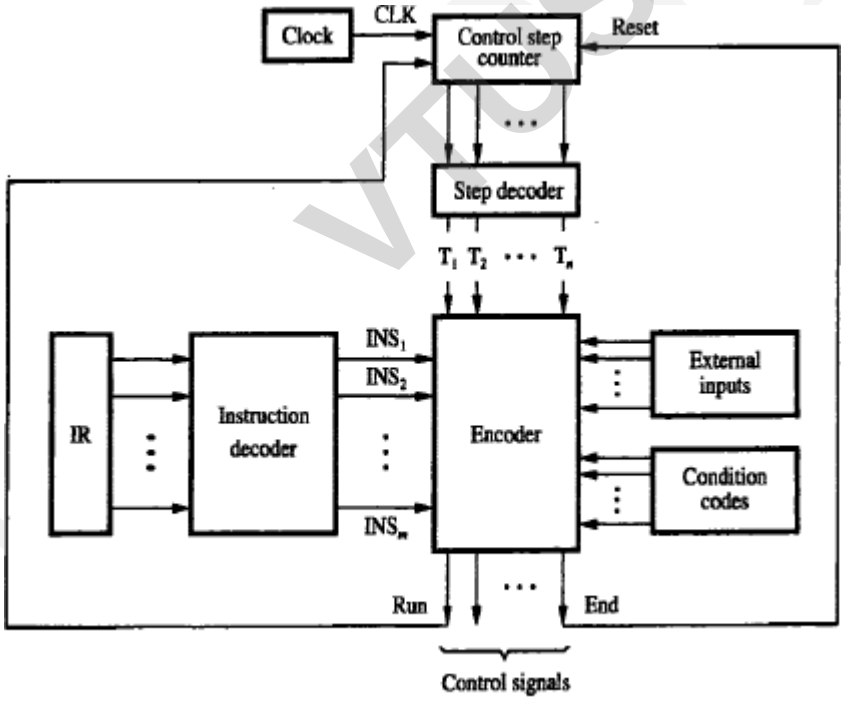


Figure 7.11 Separation of the decoding and encoding functions.

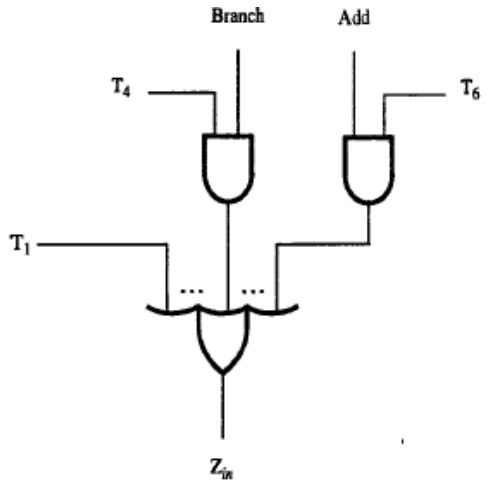
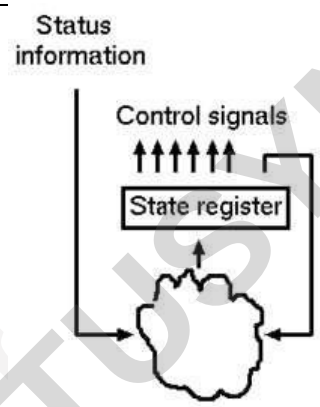
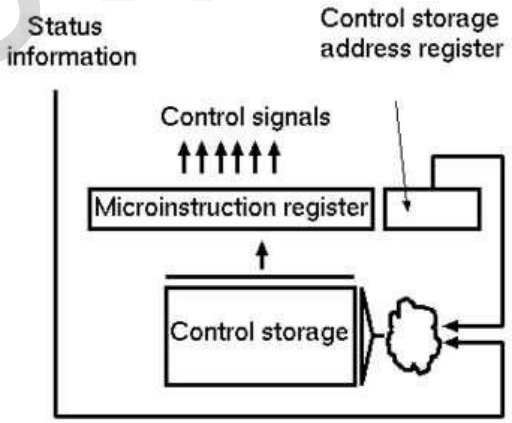


Figure 7.12 Generation of the Z_{in} control signal

HARDWIRED CONTROL VS MICROPROGRAMMED CONTROL

Attribute	Hardwired Control	Microprogrammed Control
Definition	Hardwired control is a control mechanism to generate control-signals by using gates, flip- flops, decoders, and other digital circuits.	Micro programmed control is a control mechanism to generate control-signals by using a memory called control store (CS), which contains the control-signals.
Speed	Fast	Slow
Control functions	Implemented in hardware.	Implemented in software.
Flexibility	Not flexible to accommodate new system specifications or new instructions.	More flexible, to accommodate new system specification or new instructions redesign is required.
Ability to handle large or complex instruction sets	Difficult.	Easier.
Ability to support operating systems & diagnostic features	Very difficult.	Easy.
Design process	Complicated.	Orderly and systematic.
Applications	Mostly RISC microprocessors.	Mainframes, some microprocessors.
Instructionset size	Usually under 100 instructions.	Usually over 100 instructions.
ROM size	-	2K to 10K by 20-400 bit microinstructions.
Chip area efficiency	Uses least area.	Uses more area.
Diagram		

MICROPROGRAMMED CONTROL

• Microprogramming is a method of control unit design (Figure 7.16).
• Control-signals are generated by a program similar to machine language programs.
• **Control Word(CW)** is a word whose individual bits represent various control-signals (like Add, PC_{in}).
• Each of the control-steps in control sequence of an instruction defines a unique combination of 1s & 0s in CW.
• Individual control-words in microroutine are referred to as **microinstructions** (Figure 7.15).
• A sequence of CWs corresponding to control-sequence of a machine instruction constitutes the **microroutine.**
• The microroutines for all instructions in the instruction-set of a computer are stored in a special memory called the **Control Store (CS)**.
• Control-unit generates control-signals for any instruction by sequentially reading CWs of corresponding microroutine from CS.
• **µPC** is used to read CWs sequentially from CS. (µPC→ Microprogram Counter).
• Every time new instruction is loaded into IR, o/p of **Starting Address Generator** is loaded into µPC.
• Then, µPC is automatically incremented by clock;
causing successive microinstructions to be read from CS.
Hence, control-signals are delivered to various parts of processor in correct sequence.

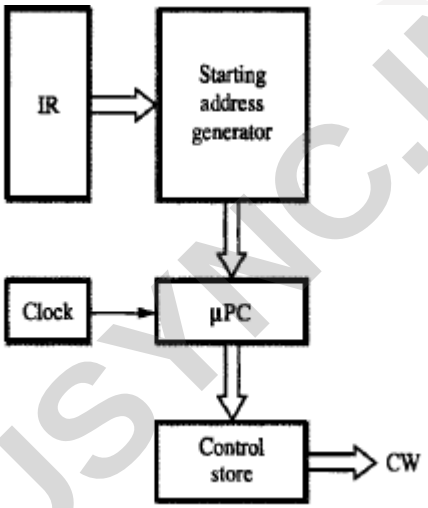


Figure 7.16 Basic organization of a microprogrammed control unit.

Micro-instruction	..	PC_{in}	PC_{out}	MAR_{in}	Read	MDR_{out}	IR_{in}	Y_{in}	Select	Add	Z_{in}	Z_{out}	$R1_{out}$	$R1_{in}$	$R3_{out}$	WMFC	End	..
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	
2		1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	

Figure 7.15 An example of microinstructions for Figure 7.6.

Advantages

• It simplifies the design of control unit. Thus it is both, cheaper and less error prone implement.
• Control functions are implemented in software rather than hardware.
• The design process is orderly and systematic.
• More flexible, can be changed to accommodate new system specifications or to correct the design errors quickly and cheaply.
• Complex function such as floating point arithmetic can be realized efficiently.

Disadvantages

• A microprogrammed control unit is somewhat slower than the hardwired control unit, because time is required to access the microinstructions from CM.
• The flexibility is achieved at some extra hardware cost due to the control memory and its access circuitry.

ORGANIZATION OF MICROPROGRAMMED CONTROL UNIT TO SUPPORT CONDITIONAL BRANCHING

• **Drawback of previous Microprogram control:**

- It cannot handle the situation when the control unit is required to check the status of the condition codes or external inputs to choose between alternative courses of action.

Solution:

- Use conditional branch microinstruction.

• In case of conditional branching, microinstructions specify which of the external inputs, condition-codes should be checked as a condition for branching to take place.

• **Starting and Branch Address Generator Block** loads a new address into µPC when a microinstruction instructs it to do so (Figure 7.18).

• To allow implementation of a conditional branch, inputs to this block consist of

→ external inputs and condition-codes &

→ contents of IR.

• µPC is incremented every time a new microinstruction is fetched from microprogram memory except in following situations:

1) When a new instruction is loaded into IR, µPC is loaded with starting-address of microroutine for that instruction.

2) When a Branch microinstruction is encountered and branch condition is satisfied, µPC is loaded with branch-address.

3) When an End microinstruction is encountered, µPC is loaded with address of first CW in microroutine for instruction fetch cycle.

Address	Microinstruction
0	PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
1	Z_{out}, PC_{in}, Y_{in}, WMFC
2	MDR_{out}, IR_{in}
3	Branch to starting address of appropriate microroutine
25	If N=0, then branch to microinstruction 0
26	Offset-field-of-IR_{out}, SelectY, Add, Z_{in}
27	Z_{out}, PC_{in}, End

Figure 7.17 Microroutine for the instruction Branch < 0.

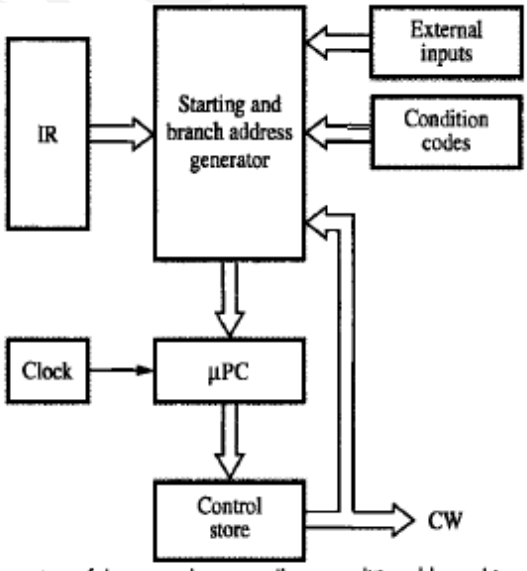


Figure 7.18 Organization of the control unit to allow conditional branching in the microprogram.

Problem 1:
Why is the Wait-for-memory-function-completed step needed for reading from or writing to the main memory?
Solution:
The WMFC step is needed to synchronize the operation of the processor and the main memory.

Problem 2:
For the single bus organization, write the complete control sequence for the instruction: Move (R1), R1
Solution:

1) PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2) Z_{out}, PC_{in}, Y_{in}, WMFC
3) MDR_{out}, IR_{in}
4) $R1_{out}$, MAR_{in}, Read
5) MDR_{inE}, WMFC
6) MDR_{out}, $R2_{in}$, End

Problem 3:

Write the sequence of control steps required for the single bus organization in each of the following instructions:

a) Add the immediate number NUM to register R1.

b) Add the contents of memory-location NUM to register R1.

c) Add the contents of the memory-location whose address is at memory-location NUM to register R1.

Assume that each instruction consists of two words. The first word specifies the operation andN the addressing mode, and the second word contains the number NUM

Solution:

(*a*) 1. PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
2. Z_{out}, PC_{in}, Y_{in}, WMFC
3. MDR_{out}, IR_{in}
4. PC_{out}, MAR_{in}, Read, Select4, Add, Z_{in}
5. Z_{out}, PC_{in}, Y_{in}
6. $R1_{out}$, Y_{in}, WMFC
7. MDR_{out}, SelectY, Add, Z_{in}
8. Z_{out}, $R1_{in}$, End

(*b*) 1-4. Same as in (*a*)
5. Z_{out}, PC_{in}, WMFC
6. MDR_{out}, MAR_{in}, Read
7. $R1_{out}$, Y_{in}, WMFC
8. MDR_{out}, Add, Z_{in}
9. Z_{out}, $R1_{in}$, End

(*c*) 1-5. Same as in (*b*)
6. MDR_{out}, MAR_{in}, Read, WMFC
7-10. Same as 6-9 in (*b*)

Problem 4:

Show the control steps for the Branch on Negative instruction for a processor with three-bus organization of the data path

Solution:

1 PC_{out}, R=B, MAR_{in}, Read, IncPC

2 WMFC

3 MDR_{outB}, R=B, IR_{in}

4. PC_{out}, Offset field of IR_{out}, Add, If N = 1 then PC_{in}, End